

DOCUMENTO DE IDEALIZACAO E FUNDACAO

BipeSend

Plataforma multitenant de CRM, atendimento, automacao, IA, catalogo e paginas publicadas.

Versao 0.2 - reestruturação modular para iniciar o produto e orientar agentes de IA

Este documento transforma a visao inicial em uma base construível. Ele separa decisoes firmes, hipoteses, riscos, regras de implementacao e um primeiro taskboard. Dominios, marca, precos e provedores ainda precisam de validacao comercial, juridica e operacional.

The mockup shows a login interface for BipeSend. On the left, a dark blue sidebar contains the BipeSend logo, the tagline 'CRM, atendimento e automacao', and a description of the platform as a workspace for transforming conversations and sales with tenant security. A small badge at the bottom of the sidebar states 'Dados protegidos por design'. On the right, a white login panel titled 'Entrar no painel' includes a sub-header 'Acesse o ambiente da sua empresa'. It features two input fields: 'E-mail profissional' (with a placeholder 'voce@empresa.com') and 'Senha' (with a placeholder 'Sua senha'). Below these is a blue 'Entrar' button. At the bottom of the panel are two links: 'Esqueci minha senha' and 'Criar conta'.

BipeSend
CRM, atendimento e automacao

Um espaço de trabalho para transformar conversas em relacionamento e vendas, com segurança por tenant.

Dados protegidos por design

Entrar no painel

Acesse o ambiente da sua empresa

E-mail profissional

Senha

Entrar

[Esqueci minha senha](#) [Criar conta](#)

Mockup 1 - primeira direcao visual para o login do painel do contratante.

Sumario

Sumario	2
1. Visao do produto	4
O que o produto precisa resolver	4
Decisoes propostas para iniciar	4
2. Superficies, dominios e arquitetura	5
Arquitetura de alto nivel	5
Estrutura de pastas proposta	6
Estrutura modular e ordem cronologica	6
3. Tenant, identidade e hierarquia	7
Modelo minimo	7
Hierarquia proposta	7
RLS e defesa em profundidade	7
4. Seguranca e protecao de dados	8
Chaves e APIs no banco	8
Risco do WhatsApp nao oficial	8
5. Stack e decisoes de implementacao	9
Redis/BullMQ ou somente PostgreSQL?	9
Node.js e Python para IA	9
6. IA, RAG e MCP com seguranca	10
Pipeline RAG	10
Guardrails obrigatorios	10
MCP interno	10
7. Modulos de negocio	11
CRM e inbox	11
Automacoes	12
Catalogo e paginas	12
8. Planos, billing e gateways	13
Entitlements	13
Stripe Connect e Mercado Pago	13
9. Direcao visual, UX, acessibilidade e SEO	14
Direcao visual	14
Tipografia e responsividade	14
Acessibilidade e SEO	14
10. Primeiro desenvolvimento: login e painel do contratante	15

Fluxo funcional	15
Critérios de aceite do primeiro marco	15
Não fazer ainda	15
11. Roadmap e taskboard	16
Sprint 1	16
12. Como usar a base com ChatGPT, Gemini, Claude e outras IAs	17
Comando principal	17
Comandos auxiliares	17
Prompt de handoff recomendado	17
13. Riscos, perguntas abertas e referencias	18
Riscos que precisam de decisao	18
Referencias tecnicas consultadas	18
Conclusao de fase	18

1. Visao do produto

O BipeSend sera uma plataforma SaaS multitenant para empresas que precisam organizar conversas, contatos, funis, automacoes, catalogos e paginas publicadas em um mesmo ambiente. A proposta e conectar atendimento e venda sem transformar a IA em uma camada com acesso irrestrito.

O que o produto precisa resolver

- dar ao contratante um painel simples para equipe, setores, cargos, conversas e oportunidades
- centralizar canais de mensagem com um provider adapter que permita trocar a integracao no futuro
- permitir funis, tags, campos e automacoes configuraveis sem criar uma regra nova por cliente
- oferecer IA com contexto da empresa, mas com tenant isolation, ferramentas limitadas e aprovacao humana
- unificar catalogo, pedidos, paginas e tracking em planos com limites mensuraveis
- separar painel tenant, superpainel e landing publica por dominios e politicas diferentes.

Decisoões propostas para iniciar

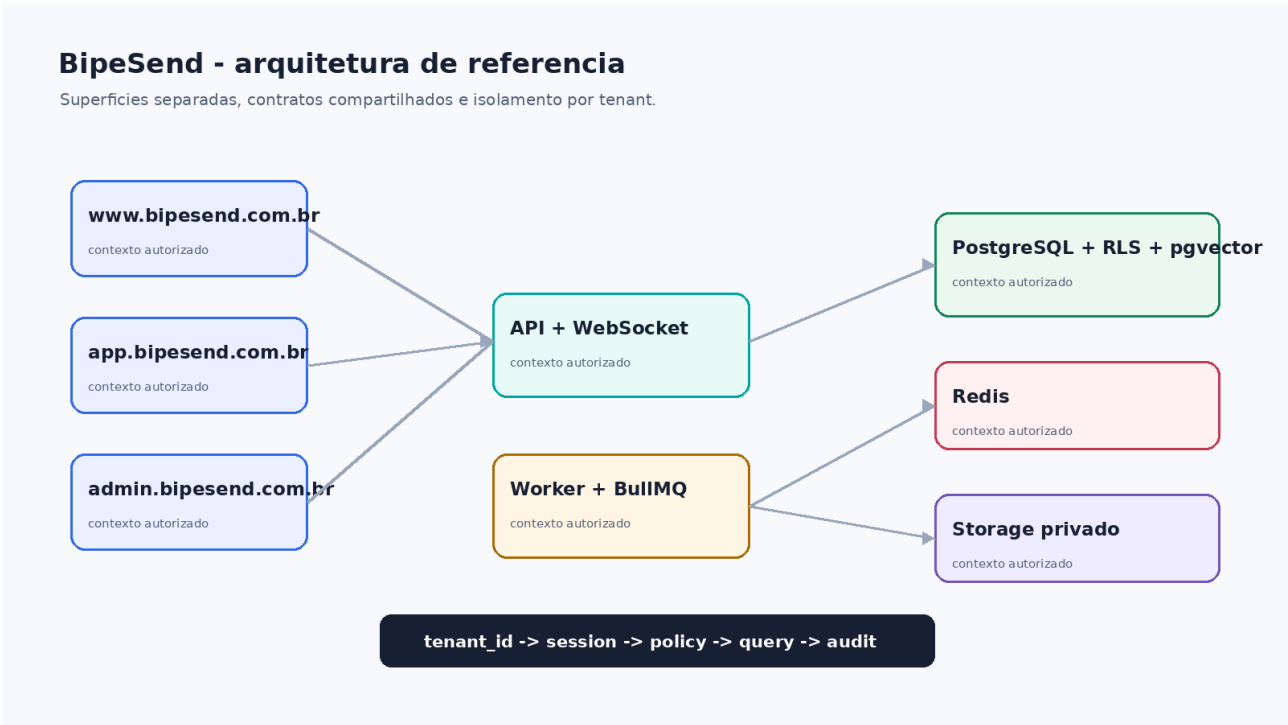
Tema	Proposta v0.2	Status
Nome	BipeSend	nome escolhido; validar marca e dominios
Frontend	Next.js, React, TypeScript, Tailwind e UI acessivel	decidido para MVP
Backend	NestJS com Fastify, REST v1 e WebSocket	decidido para MVP
Dados	PostgreSQL, RLS, migracoes SQL/Drizzle e pgvector	decidido para MVP
Filas	Redis + BullMQ com outbox no PostgreSQL	decidido para MVP
IA	microservico Python/FastAPI; Node como autoridade	decidido para MVP
WhatsApp	adapter isolado; provider nao oficial apenas em ambiente controlado	risco alto; validar antes de vender

2. Superficies, dominios e arquitetura

A separacao por dominio facilita politicas de cookie, observabilidade, deploy, SEO e controle de acesso. O frontend pode compartilhar tokens e componentes, mas cada superficie deve ter seu contexto de autenticao.

Superficie	Dominio candidato	Responsabilidade
Dominio primario	bipesend.com.br	mercado brasileiro; registrar apos validacao
Dominio secundario	bipesend.com	reserva/redirecionamento planejado
Landing	www.bipesend.com.br	marketing, SEO, cadastro e checkout
Painel tenant	app.bipesend.com.br	CRM, inbox, equipe, billing e configuracoes
Superpainel	admin.bipesend.com.br	tenants, planos, regras, integracoes e suporte
API	api.bipesend.com.br	REST, WebSocket, auth e contratos
Webhooks	hooks.bipesend.com.br	callbacks de provedores com idempotencia
Publicacao	tenant-slug.bipesend.com.br	landing/catalogo do contratante

Arquitetura de alto nivel



Mockup 2 - o tenant atravessa toda a cadeia: sessao, policy, query e auditoria.

O caminho de dados deve ser previsivel: browser -> API -> autenticao -> TenantContext -> autorizacao -> caso de uso -> repositorio -> PostgreSQL. Workers recebem jobs tipados; a IA chama somente ferramentas registradas. Nenhuma tela deve acessar banco ou provider diretamente.

Estrutura de pastas proposta

```
apps/  
tenant-web/ painel app.bipesend.com.br  
superadmin-web/ painel admin.bipesend.com.br  
marketing-web/ landing www.bipesend.com.br  
api/ NestJS/Fastify  
worker/ BullMQ, webhooks, automacoes e mensagens  
ai-worker/ jobs assincronos de ingestao e avaliacao  
services/  
ai-service/ microservico Python/FastAPI para RAG e LLM  
packages/  
ui/ db/ auth/ contracts/ events/ security/ config  
rules/ regras lidas por agentes de IA  
docs/ taskboard, ADRs e arquitetura  
infra/ Docker, proxy, migrations operacionais  
scripts/ checks e geracao de artefatos
```

Estrutura modular e ordem cronologica

```
apps/api/src/modules/  
00-shared/ 01-identity/ 02-tenancy/ 03-authorization/  
04-team/ 05-crm/ 06-inbox/ 07-messaging/  
08-automation/ 09-knowledge/ 10-catalog/ 11-pages/  
12-billing/ 13-integrations/ 14-platform/ 99-test-support/  
  
Cada modulo: domain/ application/ infrastructure/ presentation/ tests/  
Cada frontend: app/ features/ components/ lib/ styles/ tests/
```

A numeracao representa ordem de dependencia e construcao. Arquivos de codigo nao recebem numeros artificiais; usam nomes semanticos e unicos, como `create-tenant.use-case.ts` e `tenant-http.controller.ts`. Arquivos acima de 300 linhas exigem justificativa e acima de 500 devem ser divididos, salvo excecoes documentadas.

3. Tenant, identidade e hierarquia

Tenant não é apenas um filtro no frontend. É um contexto de segurança que deve ser propagado pela sessão, caso de uso, banco, fila, storage, cache, logs e IA.

Modelo mínimo

Entidade	Papel	Regra
tenants	empresa contratante	estado, slug, plano e configurações
users	identidade global	não guarda dados operacionais de tenant
memberships	vínculo user-tenant	status, cargo e preferência
roles	cargo configurável	nunca acima do criador/tenant_admin
permissions	capacidade canônica	chaves estáveis, sem nomes livres
audit_logs	trilha de segurança	actor, tenant, recurso e resultado

Hierarquia proposta

- platform_owner: equipe proprietária, em superfície e autenticação separadas.
- tenant_admin: pessoa que contratou; controla o tenant e não pode ser removida por manager.
- manager: cargo configurável, limitado pelas permissões que possui.
- agent: opera inbox/CRM dentro do escopo atribuído.
- viewer: leitura limitada.

O frontend nunca decide se alguém é admin. O backend calcula a autorização com RBAC + ABAC: permissão, tenant, setor, recurso, estado e ação. O superadmin pode ter acesso de suporte somente com motivo, prazo e auditoria.

RLS e defesa em profundidade

```
BEGIN;  
SELECT set_config('app.tenant_id', :tenant_id, true);  
SELECT set_config('app.user_id', :user_id, true);  
-- repositório executa query tenant-owned;  
-- PostgreSQL aplica policy RLS e FORCE ROW LEVEL SECURITY;  
COMMIT;
```

RLS não substitui autorização de negócio. Ele reduz o impacto de um bug de query, enquanto a camada de aplicação decide se a pessoa pode ver, editar, atribuir ou exportar aquele recurso.

4. Seguranca e protecao de dados

Seguranca sera tratada como requisito de produto. O primeiro threat model deve cobrir IDOR/cross-tenant, sequestro de conta, upload malicioso, XSS/CSRF/SSRF, webhook forjado, abuso de filas, prompt injection, exfiltracao e segredo exposto.

Area	Controle minimo
Senha e sessao	Argon2id, cookie HttpOnly/Secure/SameSite, rotacao e invalidacao
Segredos	secret manager; envelope encryption AES-256-GCM; chave mestra fora do banco
API	schemas, limite, timeout, idempotencia, erros seguros e audit trail
Webhooks	assinatura, timestamp, raw body quando necessario, dedupe e retry
Uploads	quarentena, MIME allowlist, limite, scan, storage privado e URL assinada
IA	tools allowlist, read-only inicial, sem SQL/codigo/filesystem e com limites
WhatsApp	opt-in, supressao, rate limit, pausa em erro e provider isolado
Operacao	logs sem segredos, dependency scan, backups e restore drill

Chaves e APIs no banco

O usuario pode cadastrar uma integracao no painel, mas o browser envia o segredo somente para um endpoint server-side. O backend cifra o valor com uma chave de envelope versionada, grava apenas ciphertext/nonce/tag/key_version e devolve ao frontend somente estado e mascara. Workers descriptografam apenas no momento do uso e nunca registram o valor.

Risco do WhatsApp nao oficial

A integracao nao oficial pode sofrer instabilidade, logout, bloqueio ou incompatibilidade. O produto nao deve prometer anti-bloqueio. A arquitetura deve permitir desligar esse provider e conectar uma alternativa oficial sem reescrever CRM, inbox, automacoes ou billing.

5. Stack e decisoes de implementacao

Camada	Escolha	Por que
UI	Next.js + React + TypeScript	SEO na landing, componentes compartilhados e boa experiencia responsiva
Estilo	Tailwind + tokens + componentes acessiveis	consistencia sem CSS espalhado
API	NestJS + Fastify	modulos, pipes, guards, WebSocket e contratos claros
ORM/migrations	Drizzle + SQL explicito para RLS	controle fino de transacoes, indices e policies
Banco	PostgreSQL + pgvector	estado, filtros tenant e embeddings no mesmo sistema
Filas	Redis + BullMQ	jobs, retry, delay, concorrencia e observabilidade
IA	Python/FastAPI separado; Node orquestra negocio	RAG/LLM isolado sem misturar autorizacao
Local	Docker Compose + Mailpit + MinIO + ngrok	callbacks e testes sem credenciais reais
CI	GitHub Actions	checks e protecao de branch

Redis/BullMQ ou somente PostgreSQL?

Para o MVP, Redis + BullMQ e a escolha operacional para filas e agendamentos. PostgreSQL continua sendo a fonte de verdade, com outbox transacional. Se a operacao crescer ou Redis deixar de ser conveniente, o contrato de job permanece e o backend de fila pode ser trocado sem mover o estado do negocio.

Node.js e Python para IA

Node.js continua sendo o nucleo de identidade, tenant, regras de negocio, WebSockets, billing, webhooks, filas e envio. O Python/FastAPI roda como microservico interno: recebe do Node apenas o historico autorizado, contexto assinado, politica e request id; consulta RAG e provedores LLM; devolve resposta, fontes, confianca e propostas de ferramenta. Ele nao grava no CRM, nao envia mensagem e nao executa codigo.

6. IA, RAG e MCP com seguranca

O tenant podera subir manuais, produtos, regras e textos. O sistema transforma esses documentos em conhecimento pesquisavel, mas deve tratar todo texto como dado nao confiavel. Uma instrucao maliciosa dentro de um PDF nao pode alterar as politicas do agente.

Pipeline RAG

```
upload privado
-> quarentena e antiviruss
-> extracao/normalizacao
-> chunks com document_id, versao e tenant_id
-> embeddings via provider autorizado
-> pgvector + metadados
-> retrieval filtrado por tenant/status/permissoes
-> prompt com fontes e politica
-> resposta com incerteza e auditoria
```

Guardrails obrigatorios

- modelo nao recebe credenciais, SQL, shell, filesystem ou acesso direto ao banco
- ferramentas sao allowlist, versionadas, tipadas e limitadas por tenant e permissoes
- envio de mensagem, desconto, alteracao de pedido ou exclusao exigem aprovacao quando a politica definir
- limites de tokens, custo, tempo e chamadas protegem plano e disponibilidade
- logs usam mascaramento e retencao definida; documentos despublicados nao entram no retrieval
- cada resposta pode carregar referencias internas para a empresa revisar a origem.

MCP interno

MCP sera um gateway de ferramentas, nao um passe livre para plugins externos. No MVP, somente ferramentas read-only como consulta de catalogo, lookup de contato e resumo de conversa. O gateway valida OAuth/identidade quando aplicavel, schema, tenant, permissoes, rate limit, origem e auditoria. Servidores MCP arbitrarios fornecidos por clientes ficam fora do MVP.

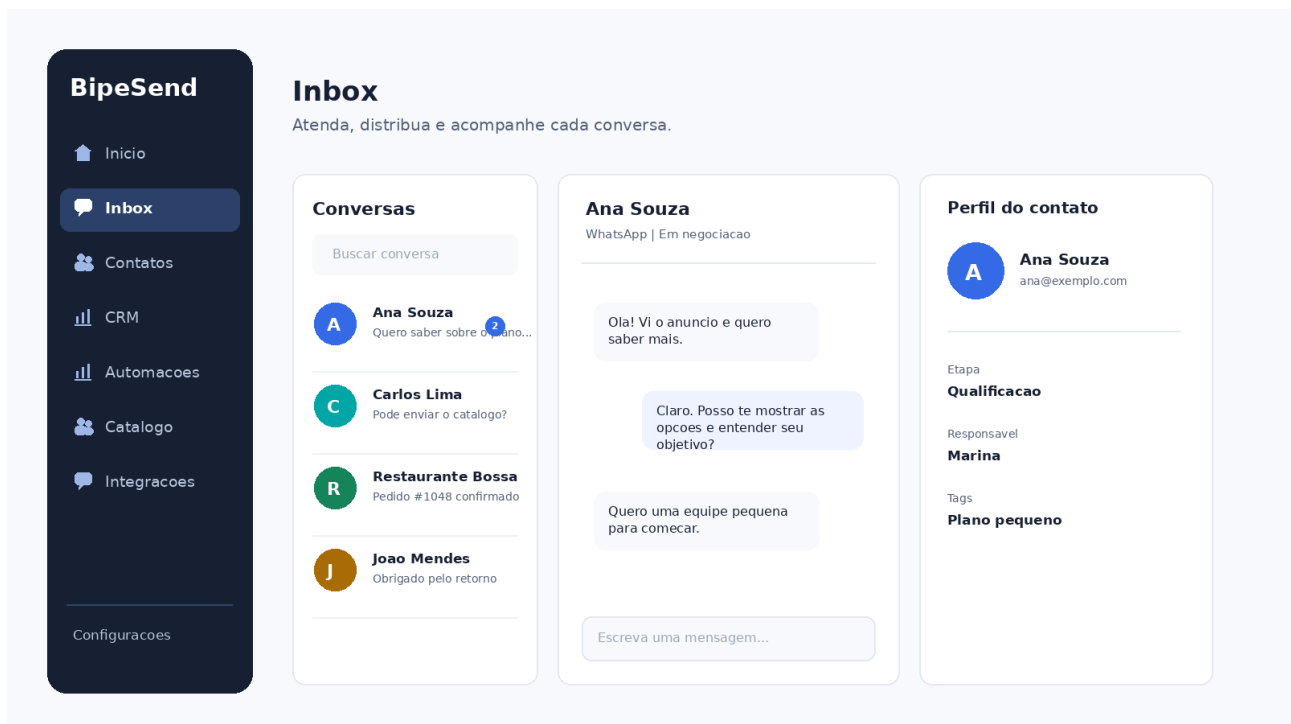
7. Modulos de negocio



Mockup 3 - painel claro, compacto e orientado a metricas explicaveis.

CRM e inbox

O painel deve permitir lista, kanban e inbox sem duplicar a fonte de dados. Cards de funil e conversas apontam para entidades canonicas. Atribuicao pode ser por setor, cargo ou colaborador, mas cada acao respeita RBAC/ABAC e registra auditoria.



Mockup 4 - inbox com lista, conversa e perfil completo do contato.

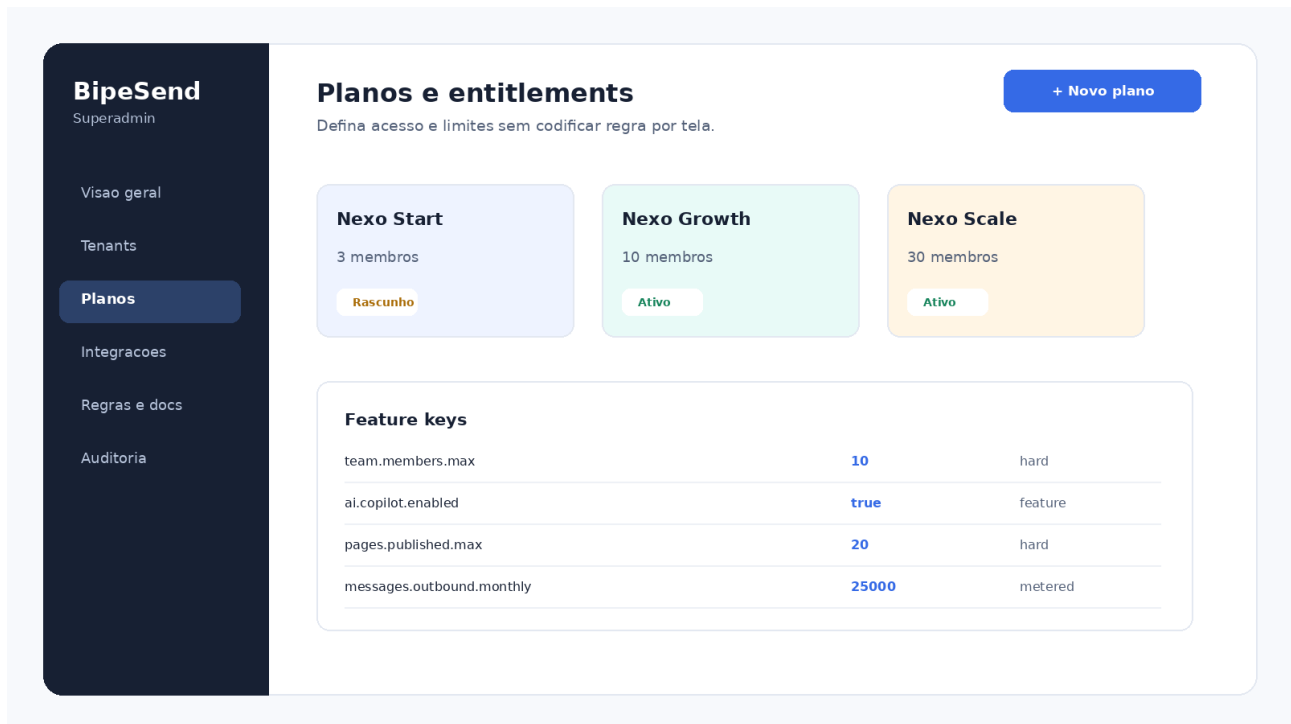
Automacoes

Automacoes sao grafos/DSL versionados com gatilho, condicao, acao, janela, timezone, limite e historico. O editor nao gera codigo executavel. Um workflow pode mover card, aplicar tag, criar tarefa, agendar mensagem ou pedir aprovacao; a execucao passa por fila e policy.

Catalogo e paginas

Catalogo deve ter categorias, produtos, opcoes, imagens, videos, disponibilidade e pedidos. O editor de paginas comeca com blocos tipados e temas, nao com HTML/JS livre. Cada pagina publicada tem slug, SEO, alt text, canonical, pixel allowlist e rollback.

8. Planos, billing e gateways



Mockup 5 - superpainel para planos e feature keys configuraveis.

Entitlements

O plano e um conjunto de feature keys. A API calcula se a empresa pode usar uma funcionalidade e qual limite se aplica. O frontend nao deve possuir ifs de negocio baseados em nome de plano; ele consome entitlements tipados.

Plano de trabalho	Perfil	Limites ilustrativos
Nexo Start	pequena equipe	3 membros, 1 numero, CRM basico
Nexo Growth	vendas em crescimento	10 membros, 3 numeros, automacoes e copiloto
Nexo Scale	operacao multicanal	30 membros, 10 numeros, RAG, catalogo e paginas
Nexo Enterprise	contrato customizado	limites, integracoes e suporte negociados

Stripe Connect e Mercado Pago

A integracao inicial deve ser sandbox-first. O superadmin cadastra credenciais de plataforma no backend; o tenant instala o gateway por OAuth. O callback valida state/PKCE quando aplicavel, usa redirect allowlist e salva tokens criptografados. Webhooks sao a fonte de verdade para estado de pagamento, com assinatura e idempotencia.

Upgrade pode liberar limites apos confirmacao do provedor. Downgrade deve valer no proximo ciclo por padrao e nunca apagar dados acima do novo limite; bloqueia novas criacoes e oferece ajuste/exportacao.

9. Direcao visual, UX, acessibilidade e SEO

Direcao visual

Painel claro, moderno, minimalista e denso sem ser apertado. Sidebar de 240 px, topbar enxuta, cards com raio 10-12 px, sombras discretas e hierarquia tipografica moderada. A interface deve orientar uma proxima acao, nao exibir numeros sem contexto.

Token	Valor proposto	Uso
brand.600	#356AE6	acao primaria
ink.900	#172033	texto principal
ink.600	#536078	texto secundario
surface.50	#F7F9FC	fundo
border.200	#E3E8F1	divisorias
success.600	#16845B	sucesso
danger.600	#C23B52	erro

Tipografia e responsividade

Inter auto-hospedada para UI e Manrope opcional para marketing, com fallback system-ui. Corpo 14-16 px, escala 12/14/16/18/20/24/28/32/40, line-height 1.4-1.55. Breakpoints praticos: 360, 768, 1024 e 1440 px. Inbox e CRM devem funcionar em 360 px.

Acessibilidade e SEO

Alvo WCAG 2.2 AA: labels, foco visivel, teclado, contraste, dialogs corretos, reduced motion, live regions e alternativa ao drag-and-drop. Landing e sites publicados usam title, description, canonical, sitemap, Open Graph, schema quando aplicavel, headings semanticos, alt text e performance. Painel autenticado usa noindex.

10. Primeiro desenvolvimento: login e painel do contratante

O primeiro incremento deve ser pequeno, completo e testavel. A experiencia começa na tela de login, mas o trabalho inclui o contrato de sessao, verificacao de e-mail, recuperacao, tenant context, autorizacao e o shell responsivo.

Fluxo funcional

1. usuario abre app.bipesend.com.br
2. envia e-mail e senha
3. API valida credenciais e estado da conta
4. API cria sessao server-side e cookie seguro
5. app carrega tenant ativo e membership
6. layout mostra apenas rotas autorizadas
7. logout invalida sessao no servidor
8. esqueci senha usa token unico e expiracao curta

Critérios de aceite do primeiro marco

- registro nao cria tenant sem verificar e-mail
- senha nunca aparece em logs ou respostas
- sessao nao e armazenada em localStorage
- usuario suspenso nao entra e recebe resposta segura
- manager nao consegue elevar cargo, remover tenant_admin ou ler outro tenant
- RLS e teste de cross-tenant falham fechado
- recuperacao invalida token apos uso e limita reenvio
- telas funcionam por teclado e em 360 px
- CI roda lint, typecheck, testes e secret scan.

Nao fazer ainda

Nao acoplar o login a WhatsApp, IA, Stripe, Mercado Pago ou editor de paginas. Essas integracoes dependem de identidade e tenant estaveis; acopla-las cedo aumenta risco e retrabalho.

11. Roadmap e taskboard

O taskboard completo esta em `docs/taskboard.md` e o mapa de pastas em `docs/module-map.md`. A construçao começa por FND-007/FND-008, depois ambiente local e identidade; CRM, WhatsApp, IA, catalogo, paginas e billing entram depois da fundacao.

Fase	Entrega	Dependencia principal
0	regras, ADRs, Git e threat model	decisao de produto
1	Docker, config, health, logs e CI	fundacao local
2	login, tenant, RBAC e shell	primeiro marco
3	equipe, setores e auditoria	identity
4	CRM, inbox e tempo real	team + contracts
5	filas, WhatsApp e automacoes	inbox + provider adapter
6	RAG, copiloto e MCP interno	documentos + policy
7	catalogo e pages builder	CRM + entitlements
8	billing e gateways	planos + webhooks
9	operacao, pentest e mobile	sistema estabilizado

Sprint 1

Começar por FND-007/008, depois INF-001/002/003, AUTH-001/004, AUTH-002/003/005, AUTH-006/008/009 e finalizar com AUTH-010/011/012. O resultado é um skeleton modular e um painel de contratante com login seguro e tenant isolation testada.

12. Como usar a base com ChatGPT, Gemini, Claude e outras IAs

A pasta `rules/` e a fonte de verdade operacional do repositório. O PDF ajuda humanos a entenderem o produto; o agente deve ler os arquivos Markdown, o taskboard e as decisões antes de editar.

Comando principal

```
!construibase

Leia rules/00_MASTER.md e todos os arquivos de regras na ordem.
Leia docs/taskboard.md e docs/decisions.md.
Leia docs/module-map.md.
Se nenhuma tarefa for informada, apenas valide a fundacao e indique o proximo item READY.
Inspecione o modulo solicitado.
Proponha arquivos, riscos e testes.
Implemente a menor mudanca segura.
Rode checks e atualize taskboard/decisoese.
```

Comandos auxiliares

Comando	Finalidade
!planejar	plano sem editar codigo; entidades, permissoes, eventos e testes
!auditar	procurar falhas de tenant, segredo, acessibilidade e contrato
!migrar	propor migration, RLS, indices, backfill e rollback

Prompt de handoff recomendado

```
Use !construibase para implementar AUTH-010.
Objetivo: tela de login do tenant-web.
Nao implemente WhatsApp, IA ou pagamentos.
Respeite rules/04_SECURITY.md, 05_AUTH_RBAC.md,
12_UX_UI.md, 13_ACCESSIBILITY_SEO.md e 14_DESIGN_TOKENS.md.
Entregue testes de estados de erro, teclado e sessao.
```

O superpainel pode exibir e versionar a documentacao, mas o código em Git e a revisao por pull request continuam sendo a fonte de verdade para mudancas tecnicas. A IA nao deve ser autorizada a transformar o painel de regras em um mecanismo de execucao irrestrita.

13. Riscos, perguntas abertas e referencias

Riscos que precisam de decisao

- validar marca e disponibilidade dos dominios BipeSend
- definir politica comercial, precos, impostos, limites e retencao
- avaliar juridica e operacionalmente o provider de WhatsApp nao oficial
- definir provedor de storage, secret manager, e-mail e observabilidade de producao
- definir politica de dados para provedores de IA e RAG
- definir se o editor de paginas sera somente interno ou produto vendavel no primeiro ano
- planejar SSO/MFA e requisitos empresariais depois do MVP.

Referencias tecnicas consultadas

- PostgreSQL - Row Security Policies: <https://www.postgresql.org/docs/17/ddl-rowsecurity.html>
- BullMQ - filas e conexoes: <https://docs.bullmq.io/>
- Stripe Connect OAuth: <https://docs.stripe.com/connect/oauth-standard-accounts>
- Mercado Pago OAuth: <https://www.mercadopago.com.br/developers/pt/docs/security/oauth/creation>
- pgvector: <https://github.com/pgvector/pgvector>
- Model Context Protocol: <https://modelcontextprotocol.io/specification>
- Next.js: <https://nextjs.org/docs>
- NestJS: <https://docs.nestjs.com/>

As referencias acima servem para confirmar contratos e capacidades das ferramentas. Elas nao substituem revisao de seguranca, termos de uso, privacidade, requisitos legais ou testes no ambiente do projeto.

Conclusao de fase

A plataforma pode ser grande, mas a construcao segura comeca pequena: identidade, tenant, autorizacao, auditoria e um painel consistente. Quando essa base estiver verde, os demais modulos podem crescer por contratos e filas sem perder o controle de dados.